Yazılımda Çok Katmanlı Mimari Ders Notu

C# Windows Forms & MySQL ile Uygulamalı Mimari Tasarım Rehberi

🎓 **Seviye:** Temel & Orta Seviye (Başlangıç Dostu) 🏛️ **Sıralama:** UI ➡️ Validator ➡️ Service ➡️ Repository ➡️ Database

1. Neden Katmanlı Mimariye İhtiyaç Duyarız?

Yazılıma ilk başladığımızda en kolay yol bir butona çift tıklayıp tüm kodları o butonun içine yazmaktır. Ancak butonun içine hem tasarım kodlarını, hem `if` kontrollerini, hem de SQL sorgularını yazarsak ortaya **"Spagetti Kod" (Spaghetti Code)** çıkar.

⚠️ **Spagetti Kod Neden Büyük Bir Tehlikedir?**

- **Kod Tekrarı (DRY İhlali):** Aynı SQL sorgusunu veya TC kontrolünü 5 farklı formda kullanmak gerekirse, aynı kodları kopyala-yapıştır yapmak zorunda kalırsınız.
- **Değişiklik Zorluğu:** Veritabanında bir tablonun adı veya kuralı değiştiğinde projedeki onlarca formu tek tek arayıp düzeltmeniz gerekir.
- **Okunabilirlik Sıfır:** Butonun içinde 100 satır karmaşık kod olması hata ayıklamayı ve ekip çalışmasını imkansızlaştırır.

Çözüm: Sorumlulukların Ayrılması ilkesidir *(Separation of Concerns)*. Her işi tek bir yere yığmak yerine, sistemi kendi uzmanlık alanından sorumlu "katmanlara" böleriz.

2. Günlük Hayattan Bir Benzetme: "Restoran Modeli"

Nesne Yönelimli Programlamanın (OOP) terimlerine boğulmadan önce bir restoranın işleyişini düşünelim. Restoranda kimse her işi tek başına yapmaz:

🍽️ **Restorandaki Görev Dağılımı ve Bizim Yazılım Katmanlarımız:**

- **Müşteri:** Programı kullanan son kullanıcıdır. Formu doldurur ve sonucu bekler.
- **Garson (Sunum / UI Katmanı - `UcCalisanEkle`):** Müşteriden siparişi alır, mutfağa teslim eder ve pişen yemeği müşteriye getirir. Garson asla mutfağa girip et pişirmez veya depoya inip pirinç taşımaz!
- **Sipariş Müfettişi / Gıda Denetçisi (Validator - `CalisanValidator`):** Siparişi ilk denetleyen kişidir: "*Müşteri 18 yaşından küçük mü?*", "*Boş alan bırakılmış mı?*", "*Sipariş kurallara uygun mu?*" Hata varsa sipariş mutfağa hiç girmez.
- **Aşçıbaşı / Mutfak Şefi (Servis Katmanı - `CalisanService`):** Denetimden geçen siparişin mutfaktaki tüm sürecini yönetir. İş kurallarını uygular (mükerrer sipariş kontrolü vb.), garnitürleri hazırlar ve depocuya malzeme listesini teslim eder.
- **Kiler / Depo Görevlisi (Repository - `CalisanRepository`):** Şefin onayladığı malzemeyi depodan (MySQL) getiren veya depoya yerleştiren kişidir. Arayüzü veya müşteriyi tanımaz.
- **Yemek Tepsisi / Sipariş Fişi (Model - `Calisan`):** Garson, şef ve depocu arasında taşınan standart tabaktır. İçinde veri taşır ama kendisi iş yapmaz.
- **Kiler / Depo (MySQL Veritabanı):** Tüm kayıtların kalıcı saklandığı fiziksel yerdir.

3. C#'ta Karşılaşılan Temel Kavramlar (Öğrenci Kılavuzu)

C# dilini ve katmanlı mimariyi rahatça anlayabilmek için sıkça karşımıza çıkan şu 4 kavramı bilmek gerekir:

A) Property Nedir ve Neden `{ get; set; }` Yazarız?

PHP veya geleneksel dillerde sınıflardaki değişkenleri korumak için `getAd()` ve `setAd()` şeklinde iki ayrı fonksiyon yazılır. C#'ta ise dışarıdan bakıldığında **düz bir değişken** gibi duran ama arka planda **metot** gibi çalışan **Property (Özellik)** yapısı vardır:

Açık Hali (Klasik Yol):

```
private string ad; // Gizli değişken (Field)
public string Ad   // Dışarıya açık kapı (Property)
{
    get { return ad; }       // Okunurken çalışır (PHP'deki getAd())
    set { ad = value; }      // Atanırken çalışır (PHP'deki setAd())
}
```

Kısayol Hali (Otomatik Property):
Özel bir şart yoksa C# bunu tek satırda yazmamıza izin verir:

```
public string Ad { get; set; } // get = Okumaya izin ver, set = Değiştirmeye izin ver
```

B) `static` Anahtar Kelimesi Ne İşe Yarar?

Normal sınıfları kullanmak için önce new ile nesne üretmek gerekir (Örn: `Calisan c = new Calisan();`).
Fakat **`static`** olarak tanımlanan bir sınıf veya metot (Örn: `VeritabaniBaglanti.BaglantiGetir()`) için new yapmaya gerek yoktur. Programın her yerinden doğrudan `SinifAdi.MetotAdi()` şeklinde ortak olarak çağrılabilir.

C) `using` Bloğu Neden Hayatidir?

Veritabanı bağlantısı açtığımızda bu bağlantı sunucuda yer kaplar. Eğer işimiz bittiğinde kapatmayı unutursak bir süre sonra veritabanı kilitlenir. `using (MySqlConnection conn = ...) { ... }` bloğu, kodun içinde hata çıksa bile süslü parantez bittiği anda veritabanı bağlantısını **otomatik ve güvenli şekilde kapatır.**

D) Neden SQL Sorgularında `@parametre` Kullanıyoruz?

❌ **YANLIŞ VE TEHLİKELİ (String Birleştirme):**
`"INSERT INTO calisanlar VALUES ('" + txt_ad.Text + "')"`
Kullanıcı ad kutusuna tırnak işareti veya zararlı bir SQL komutu yazarsa veritabanınız silinebilir (**SQL Injection** açığı).

✅ **DOĞRU VE GÜVENLİ (Parametreli Sorgu):**

```
cmd.CommandText = "INSERT INTO calisanlar (ad) VALUES (@ad);";
cmd.Parameters.AddWithValue("@ad", calisan.Ad);
```

MySQL gelen veriyi sadece düz metin olarak algılar; asla komut olarak çalıştırmaz.

4. Projemizdeki 5 Katman ve Görev Sıralaması

1. UI (Sunum)	**UcCalisanEkle.cs:** Form kutularındaki yazıları toplar, `Calisan` modeline doldurur ve süreci başlatır.
⬇️ 1. Form verisini toplar > İlk olarak doğrulamaya sunar	
2. Validator (Doğrulama)	**CalisanValidator.cs:** Verileri denetler (Ad boş mu, TC 11 hane mi, maaş pozitif mi?). Hata varsa işlemi anında durdurur.
⬇️ 2. Veriler kurallara uygunsa > Servis iş mantığı devreye girer	
3. Service (İş Mantığı)	**CalisanService.cs:** Süreci yönetir. TC çakışması var mı bakar, fotoğraf dosyasını klasöre kopyalar ve Repository'ye kaydettirir.
⬇️ 3. İş kuralları tamamlanınca > SQL çalıştırması için Repository'ye teslim eder	
4. Repository (Veri Erişim)	**CalisanRepository.cs:** Sadece SQL (`INSERT`) çalıştırır. MySQL veritabanına bağlanır ve veriyi satır olarak yazar.
⬇️ 4. Parametreli SQL komutu yürütür	
5. Veritabanı (DB)	**MySQL (teknik_servis):** Fiziksel verilerin tutulduğu MySQL veritabanı ve `calisanlar` tablosu.
Ortak Katman: Model	**Calisan.cs:** Katmanlar arasında veriyi taşıyan saf nesnedir (Ad, Soyad, TC, Telefon, Maas...).

5. Bir Kaydın Yolculuğu: Kod Üzerinde Adım Adım

Adım 1: Arayüz (UI) Veriyi Toplar

```
// UcCalisanEkle.cs - Butona basılınca çalışan kod:
Calisan yeniCalisan = new Calisan();
yeniCalisan.Ad = txt_calisan_adi.Text.Trim();
yeniCalisan.Tc = txt_calisan_tc.Text.Trim();
// ... diğer alanlar doldurulur ...

// Modeli kaydetmesi için doğrudan Servis katmanına teslim ediyoruz:
IslemSonucu sonuc = _calisanService.Kaydet(yeniCalisan, secilenFotoYolu);
MessageBox.Show(sonuc.Mesaj);
```

Adım 2: Servis İçinde İlk Olarak Validator Çalıştırılır

```
// CalisanService.cs - İşlemleri yöneten orkestra şefi:
public IslemSonucu Kaydet(Calisan calisan, string fotoYolu)
{
    // 1. ÖNCE VALIDATOR: Alanlar boş mu, formatlar doğru mu denetle:
    ValidationResult dogrulama = _validator.Dogrula(calisan);
    if (dogrulama.GecerliMi() == false)
        return IslemSonucu.BasarisizSonuc(dogrulama.HatalariBirlestir());

    // 2. SONRA SERVICE İŞ KURALI: Aynı TC ile daha önce kayıt yapılmış mı?
    if (_repository.TcVarMi(calisan.Tc) == true)
        return IslemSonucu.BasarisizSonuc("Bu TC kimlik numarası zaten kayıtlı!");

    // 3. EN SON REPOSITORY: Veritabanına SQL ile ekle:
    bool eklendi = _repository.Ekle(calisan, out string hata);
    return eklendi ? IslemSonucu.BasariliSonuc("Başarıyla eklendi.") : IslemSonucu.BasarisizSonuc(hata);
}
```

6. Katmanların Görev Sınırları (Kim Ne Yapar?)

Her katmanın yapmaya yetkili olduğu işler ve **asla yapmaması gereken yasaklar** şunlardır:

Katman	Ne Yapar? (İzinli Görevler)	Ne Yapamaz? (Kesinlikle Yasak!)
1. UI (Sunum)	Kullanıcıdan veri alır, ekrana buton/kutu çizer, servis sonucunu `MessageBox` ile gösterir.	❌ Asla SQL sorgusu (`INSERT`) yazamaz. Veritabanına doğrudan bağlanamaz. Doğrulama yapmaz.
2. Validator (Doğrulama)	Verilen modelin alanlarını kurallara göre inceler (boş mu, 11 hane mi, negatif mi).	❌ Veritabanına bağlanamaz. Ekrana mesaj basamaz. Sadece hata listesi döner.
3. Service (İş Mantığı)	Validator ve Repository sürecini koordine eder, iş kurallarını işletir, dosya kopyalar, sonucu paketler.	❌ Ekrana pencere çizemez, `MessageBox` açamaz. SQL sorgusu yazmaz (Repository'ye bırakır).
4. Repository (Veri Erişim)	Veritabanına bağlanır, SQL komutunu parametrelerle çalıştırır, sonucunu döner.	❌ Arayüzü bilmez. Formdaki kutucuklara (`TextBox`) doğrudan erişemez.
Model (Varlık)	Sadece veri alanlarını (`Ad`, `Soyad`, `Tc`, `Maas`) taşır.	❌ İçinde SQL sorgusu, bağlantı veya kural kodu barındırmaz. Saf veri kutusudur.
VeritabaniBaglanti	Bağlantı cümlesini tutar ve yeni bağlantı üretir.	❌ Tablo adlarını veya SQL komutlarını bilmez. Sadece bağlantı sağlar.

7. Derste Uygulama Yaparken 5 Altın Kural

1. **"Ben bu kodu hangi sınıfa yazmalıyım?" sorusunu kendinize sorun:**
   - Kullanıcı tıklaması veya ekranda mesaj gösterme işi mi? ➡️ **UI (Form)**
   - "Bu alan boş mu, 11 hane mi?" format/veri kontrolü mü? ➡️ **Validator**
   - Süreci yönetme, fotoğraf kopyalama, "Bu TC zaten var mı?" iş kuralı mı? ➡️ **Service**
   - Veritabanına bağlanıp SQL çalıştırma işi mi? ➡️ **Repository**
2. **Formlarınızı tertemiz tutun:** Buton tıklama olayının içinde sadece verileri toplayıp `_calisanService.Kaydet(...)` çağrısı yapın. Formunuz 10-15 satırı geçmesin.
3. **SQL Injection'a asla geçit vermeyin:** Sorguları + ile birleştirmek yerine daima `cmd.Parameters.AddWithValue()` kullanın.
4. **Bağlantıları using bloğuna emanet edin:** Açık bağlantı kalmaması için her zaman `using (MySqlConnection conn = ...)` kullanın.
5. **Fiziksel Klasör ≠ Mantıksal Namespace:** Dosyayı yeni klasöre (örn. `UserControls`) taşımak kodun başındaki namespace'i otomatik değiştirmez. Diğer dosyalarda `using` ile çağırmak için dosyadaki namespace'i de güncellemeli ve `.cs` ile `.Designer.cs` namespace'lerinin birebir aynı olmasına dikkat etmelisiniz.

> 🎓 **Öğretmen Notu:** Bu mimari sadece C# Windows Forms için değil; ileride öğreneceğiniz **ASP.NET Core Web API, PHP (Laravel / Symfony), Java (Spring Boot)** gibi tüm kurumsal yazılım ekosistemlerinde geçerli olan evrensel bir endüstri standardıdır.